

FIȘĂ DE LUCRU

Clasa list în Python – operatori și metode

Clasa a IX-a – Informatică (CS) – Matematica-Informatică

1. Noțiuni teoretice

Lista este o colecție ordonată și mutabilă de elemente în Python. Elementele pot fi de tipuri diferite și sunt accesibile prin index (poziție), începând de la 0.

Caracteristici principale: mutabilitate (elementele pot fi modificate după creare), acces direct prin index, păstrarea ordinii de inserare, permit elemente duplicate.

Crearea unei liste:

```
# Lista vida  
a = []  
  
# Lista cu valori initiale  
note = [9, 7, 10, 8, 6]  
  
# Lista cu elemente de tipuri diferite  
info = ['Ana', 16, 9.50, True]
```

2. Operatori ai clasei list

Operator	Descriere	Exemplu
[]	Acces la element prin index	note[0] → 9
[i:j]	Felierea (slicing) – sublista de la i la j-1	note[1:3] → [7, 10]
in	Verificare apartenență	10 in note → True
not in	Verificare non-apartenență	5 not in note → True
+	Concatenare (unire două liste)	[1,2] + [3,4] → [1,2,3,4]
*	Înmulțire (repetare)	[0] * 3 → [0, 0, 0]
==, !=	Comparare (egalitate / inegalitate)	[1,2] == [1,2] → True
<, >	Comparare lexicografică	[1,3] > [1,2] → True

3. Metode ale clasei list

Metodă	Descriere	Exemplu
append(x)	Adaugă elementul x la sfârșitul listei	note.append(5) → [9,7,10,8,6,5]
insert(i, x)	Inserează x pe poziția i	note.insert(0, 10) → [10,9,7,...]
pop(i)	Șterge și returnează elementul de pe poziția i (implicit ultimul)	note.pop() → 6
remove(x)	Șterge prima apariție a valorii x	note.remove(7) → [9,10,8,6]

index(x)	Returnează poziția primei apariții a lui x	note.index(10) → 2
count(x)	Returnează numărul de apariții ale lui x	note.count(9) → 1
sort()	Sortează lista crescător (in-place)	note.sort() → [6,7,8,9,10]
reverse()	Inversează ordinea elementelor	note.reverse() → [6,8,10,7,9]
copy()	Returnează o copie superficială a listei	b = note.copy()
extend(lst)	Adaugă elementele din lst la sfârșit	note.extend([4,3])
clear()	Elimină toate elementele	note.clear() → []

4. Exemple pas cu pas

Exemplul 1: Gestionarea unei liste de produse într-un coș de cumpărături

```
cos = ['lapte', 'paine', 'oua']
print(cos)          # ['lapte', 'paine', 'oua']

# Adaugam un produs la sfarsit
cos.append('unt')
print(cos)          # ['lapte', 'paine', 'oua', 'unt']

# Inseram un produs pe pozitia 1
cos.insert(1, 'branza')
print(cos)          # ['lapte', 'branza', 'paine', 'oua', 'unt']

# Stergem un produs dupa valoare
cos.remove('paine')
print(cos)          # ['lapte', 'branza', 'oua', 'unt']

# Verificam daca un produs este in cos
print('oua' in cos) # True
print('mere' in cos) # False

# Stergem ultimul produs adaugat
ultimul = cos.pop()
print(ultimul)      # unt
print(cos)          # ['lapte', 'branza', 'oua']
```

Exemplul 2: Prelucrarea unei liste de note

```
note = [8, 9, 7, 10, 9, 6, 8]

# Numarul de note
print('Numar note:', len(note))      # 7

# Nota minima si maxima
print('Minim:', min(note))           # 6
print('Maxim:', max(note))           # 10

# Media notelor
media = sum(note) / len(note)
print('Media:', round(media, 2))     # 8.14
```

```
# De cate ori apare nota 9?
print('Nota 9 apare de', note.count(9), 'ori') # 2

# Pe ce pozitie este prima nota de 10?
print('Prima nota 10 pe pozitia:', note.index(10)) # 3

# Sortam notele crescator
note_sortate = note.copy() # copie, ca sa pastram originalul
note_sortate.sort()
print('Sortate:', note_sortate) # [6, 7, 8, 8, 9, 9, 10]
```

Exemplul 3: Concatenare, multiplicare și comparare

```
# Concatenare
grupa_a = [9, 8, 10]
grupa_b = [7, 9, 6]
toate_notele = grupa_a + grupa_b
print(toate_notele) # [9, 8, 10, 7, 9, 6]

# Multiplicare
zerouri = [0] * 5
print(zerouri) # [0, 0, 0, 0, 0]

# Comparare
print([1, 2, 3] == [1, 2, 3]) # True
print([1, 2, 3] == [3, 2, 1]) # False
print([1, 3] > [1, 2]) # True (comparare lexicografica)
```

5. Erori frecvente

Eroare 1: IndexError – acces la un index inexistent

Greșit:

```
note = [9, 7, 10]
print(note[3])    # IndexError! Indicii valizi: 0, 1, 2
```

Corect:

```
print(note[2])    # 10
# sau verificam lungimea inainte:
if len(note) > 3:
    print(note[3])
```

Eroare 2: Confuzia între sort() și sorted()

Atenție: sort() modifică lista originală și returnează None. sorted() returnează o listă nouă.

```
note = [8, 5, 9]

# GRESIT:
rezultat = note.sort()    # rezultat va fi None!

# CORECT - varianta 1 (modifica lista):
note.sort()               # note devine [5, 8, 9]

# CORECT - varianta 2 (lista noua):
note_noi = sorted(note)  # note ramane neschimbata
```

Eroare 3: Copierea prin atribuire vs. copy()

```
# GRESIT - atribuirea NU creeaza o copie:
a = [1, 2, 3]
b = a          # b este ACEEASI lista ca a
b.append(4)
print(a)       # [1, 2, 3, 4] -- a s-a modificat!

# CORECT - folosim copy():
a = [1, 2, 3]
b = a.copy()   # b este o COPIE independenta
b.append(4)
print(a)       # [1, 2, 3] -- a ramane neschimbata
```

6. Exerciții

Exercițiul 1 (ușor)

Se dă lista produse = ['mere', 'pere', 'banane', 'portocale']. Scrieți instrucțiunile Python pentru: a) adăugarea valorii 'kiwi' la sfârșit; b) inserarea valorii 'mango' pe poziția 2; c) ștergerea valorii 'pere'; d) afișarea numărului de elemente din listă.

Exercițiul 2 (ușor)

Se dă lista numere = [3, 7, 2, 7, 5, 7, 1]. Determinați (folosind metode ale clasei list): a) de câte ori apare valoarea 7; b) pe ce poziție se află prima apariție a valorii 7; c) lista sortată crescător (fără a modifica lista originală).

Exercițiul 3 (mediu)

Se citesc de la tastatură n numere întregi și se memorează într-o listă. Scrieți un program care: a) afișează suma, media, minimumul și maximum; b) afișează elementele mai mari decât media; c) afișează lista în ordine inversă.

Exercițiul 4 (mediu)

Scrieți un program care gestionează o listă de participanți la un eveniment. Programul permite: adăugarea unui participant (append), înscrierea pe o poziție dorită (insert), retragerea ultimului înscris (pop), căutarea unui participant (in) și afișarea listei sortate alfabetic (sort).

Exercițiul 5 (avansat)

Se dau două liste de note ale elevilor din două grupe. Scrieți un program care: a) concatenează cele două liste; b) determină nota maximă și nota minimă din lista unificată; c) numără elevii cu note peste media generală; d) creează o copie sortată descrescător (folosind sort cu reverse=True).

Exercițiul 6 (avansat)

Scrieți un program care simulează un coș de cumpărături cu un meniu interactiv: 1) Adaugă produs, 2) Șterge produs, 3) Afișează coșul, 4) Golește coșul (clear), 5) Numără câte produse sunt în coș (len), 6) Ieșire. Folosiți o buclă while și metode ale clasei list.

7. Rezumat

- Clasa list din Python oferă o colecție ordonată și mutabilă, cu acces direct prin index.
- Operatorii permit accesul ([]), verificarea apartenenței (in), concatenarea (+), multiplicarea (*) și compararea (==, <, >).
- Metodele principale (append, insert, pop, remove, sort, copy, extend, clear) permit manipularea eficientă a elementelor.
- Atenție la diferența dintre atribuire (și cele două variabile referă aceeași listă) și copiere cu copy() (listă independentă).
- Funcțiile predefinite len(), min(), max(), sum() sunt utile pentru prelucrarea rapidă a listelor numerice.